

Agentes de IA II: Coordinación y Sistemas Avanzados

Memoria, razonamiento y multi-agentes

Cesar Garcia

2025

- **Sesión anterior:** loop del agente, prompts, herramientas básicas
- **Hoy:** patrones de razonamiento, memoria y coordinación
- **Meta:** construir un sistema multi-agente real
- **Proyecto:** análisis de gastos personales con varios agentes especializados

Un agente solo puede hacer mucho. Un equipo de agentes puede hacer más.

Base de todo lo que sigue

Observación -> Razonamiento -> Acción -> Resultado
~ |
+-----+

Hoy expandimos:

- *Razonamiento*: patrones más sofisticados (ReAct, Plan-Execute)
- *Estado*: memoria entre pasos y entre agentes
- *Acción*: delegar a otros agentes especializados

Tres enfoques principales

Patrón	Estilo	Cuándo usarlo
ReAct	Razona -> Actúa (intercalado)	Tareas exploratorias
Chain-of-Thought	Razona primero, luego actúa	Problemas complejos
Plan-Execute	Planifica -> Ejecuta pasos	Tareas estructuradas

ReAct (Reason + Act)

Interleaving de pensamiento y acción

Thought: Necesito saber el total de gastos en enero.

Action: consultar_gastos(mes="enero")

Observation: [280.5, 45.0, 30.0, 120.0, 85.5, 60.0]

Thought: Ahora calculo la suma.

Action: calcular(expresion="sum([280.5, 45.0, ...]))")

Observation: 621.0

Thought: Ya tengo el dato. Puedo responder.

Answer: En enero gastaste \$621.00 en total.

Ventaja: cada pensamiento informa la siguiente acción.

Chain-of-Thought (CoT)

Razonamiento extendido

El modelo *piensa en voz alta* antes de actuar:

```
system = """
```

```
Antes de usar cualquier herramienta, explica paso a paso  
tu plan de acción. Luego ejecuta cada paso.
```

```
Formato:
```

```
Plan: [pasos numerados]
```

```
Ejecución: [herramientas y resultados]
```

```
Respuesta: [conclusión final]
```

```
"""
```

Ventaja: el razonamiento es auditable y corregible.

Separación de fases

Fase 1 - PLANIFICACIÓN:

Input: "Analiza mis gastos del primer trimestre"

Output: Plan de 4 pasos definidos

Fase 2 - EJECUCIÓN:

Paso 1: consultar_gastos(mes="enero")

Paso 2: consultar_gastos(mes="febrero")

Paso 3: consultar_gastos(mes="marzo")

Paso 4: calcular y comparar totales

Fase 3 - SÍNTESIS:

Output: Reporte estructurado

Ventaja: el plan puede ser revisado antes de ejecutar.

Tipos de memoria

- **Memoria de contexto (corto plazo):** el historial de mensajes
 - Límite: ventana de contexto del modelo
- **Memoria de trabajo:** un diccionario que tu código mantiene
 - Persiste entre pasos sin ocupar tokens
- **Memoria externa:** base de datos, archivos, vector stores
 - Escala a millones de registros

State dict

```
estado = {  
    "gastos_enero": None,  
    "gastos_febrero": None,  
    "total_calculado": None,  
    "categorias_analizadas": [],  
}
```

El agente actualiza el estado en cada paso
Tu código lo pasa como contexto al siguiente paso

```
system = f"""Estado actual:  
{json.dumps(estado, ensure_ascii=False, indent=2)}
```

```
Usa esta información antes de llamar herramientas."""
```

Estrategias para contextos largos

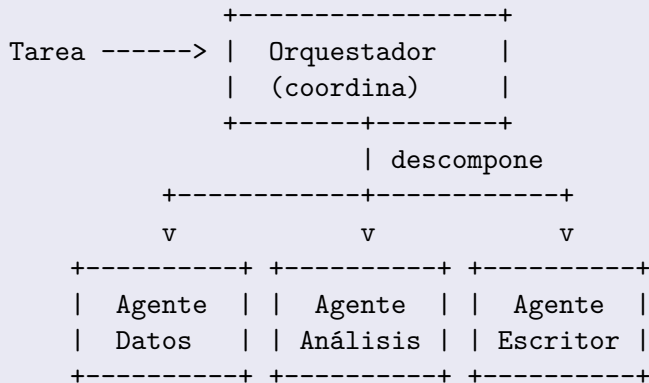
Problema: el historial crece con cada turno.

Soluciones:

- 1 **Ventana deslizante:** conservar solo los últimos N mensajes
- 2 **Resumen progresivo:** comprimir mensajes antiguos
- 3 **Memoria selectiva:** extraer y guardar solo lo importante
- 4 **Retrieval:** buscar en memoria externa solo lo relevante

El contexto es el recurso más valioso del agente. Adminístralo.

El patrón Orquestador + Subagentes



Ventajas

- **Especialización:** cada agente tiene un system prompt específico
- **Paralelismo:** subagentes pueden ejecutarse en paralelo
- **Modularidad:** fácil agregar o reemplazar subagentes
- **Contextos limpios:** cada agente solo ve lo que necesita

Desafíos:

- Comunicación entre agentes
- Manejo de errores en cadena
- Costo acumulado de tokens

Especializado en acceso

```
sistema_datos = """
```

```
Eres un agente especializado en acceso a datos.
```

```
Tu ÚNICA función es: leer, filtrar y transformar datos.
```

```
NO interpretes. NO hagas recomendaciones.
```

```
Devuelve SIEMPRE datos en formato JSON.
```

```
"""
```

Herramientas: consultar_gastos, filtrar_por_categoria, calcular_totales

Especializado en interpretación

```
sistema_analista = """
```

```
Eres un analista financiero experto.
```

```
Recibirás datos ya procesados en JSON.
```

```
Tu función es: interpretar, identificar patrones,  
comparar períodos y hacer recomendaciones.
```

```
NO accedas a datos directamente.
```

```
"""
```

Input: JSON del agente de datos **Output:** análisis en lenguaje natural

Especializado en formato

```
sistema_escritor = """
```

```
Eres un comunicador experto.
```

```
Recibirás un análisis técnico.
```

```
Tu función es: convertirlo en un reporte claro,  
bien estructurado y fácil de entender.
```

```
Usa bullet points, secciones y resaltado.
```

```
"""
```

Input: análisis del agente analista **Output:** reporte final formateado

Director de orquesta

```
def orquestador(pregunta):  
    # 1. Obtener datos (agente_datos)  
    datos = agente_datos(pregunta)  
  
    # 2. Analizar resultados (agente_analista)  
    analisis = agente_analista(datos)  
  
    # 3. Formatear reporte (agente_escritor)  
    reporte = agente_escritor(analisis)  
  
    return reporte
```

El orquestador decide el flujo. Los subagentes ejecutan.

Diseño responsable

Validación de entrada: - Sanitizar texto antes de usarlo en prompts - Limitar longitud de inputs del usuario

Validación de salida: - Verificar que la respuesta tiene el formato esperado - Detectar alucinaciones o datos inventados

Límites operacionales: - Máximo de pasos por agente (ej: 10) - Timeout por llamada - Presupuesto máximo de tokens

Fail-safe

```
MAX_PASOS = 10
pasos = 0

while respuesta.stop_reason == "tool_use":
    pasos += 1
    if pasos > MAX_PASOS:
        return "Error: límite de pasos alcanzado"
    # continuar loop...
```

Regla: todo agente debe tener una condición de salida garantizada.

Un agente sin límites es un agente impredecible.

Sistema de Análisis Financiero

Pregunta: “Dame un resumen de mis gastos del primer trimestre”

Orquestador

+-- Agente Datos:

```
|   consultar_gastos(mes="enero")  
|   consultar_gastos(mes="febrero")  
|   consultar_gastos(mes="marzo")  
|   calcular_por_categoria(datos)  
|
```

+-- Agente Analista:

```
|   identifica patrones, picos, categorías altas  
|
```

+-- Agente Escritor:

```
    reporte final con secciones y recomendaciones
```

Sistemas > Agentes individuales

Los sistemas multi-agente son:

- **Más capaces:** división del trabajo y especialización
 - **Más robustos:** errores en un agente no bloquean todo
 - **Más mantenibles:** cambiar un agente no afecta los demás
 - **Más escalables:** agregar agentes = agregar capacidades
- La inteligencia emerge de la coordinación, no de un solo modelo.*

Conceptos vistos

- Patrones de razonamiento: ReAct, CoT, Plan-Execute
- Estado y memoria: contexto, working memory, externa
- Gestión de ventana de contexto
- Patrón Orquestador + Subagentes
- Agentes especializados (datos, análisis, escritura)
- Guardrails y diseño responsable
- Sistema multi-agente completo

Tienes las herramientas para construir sistemas de IA reales.

Hacia producción

- **Frameworks:** LangChain, LlamaIndex, CrewAI, AutoGen
- **Observabilidad:** trazas, logs, métricas de costo
- **Evaluación:** cómo medir si el agente es bueno
- **Seguridad:** prompt injection, jailbreaks, datos sensibles
- **Escalado:** agentes en la nube, colas de tareas

El campo evoluciona rápidamente. Los fundamentos que aprendiste aquí son permanentes.